FloodLens

Computational Imaging and Augmented Reality for Urban Flood Risk Visualization

Hasnain — Independent Research Project

25th August, 2026

Abstract

Flooding is a recurring problem in Karachi and in many other South Asian cities, and most of the tools built to study it computationally are trained on imagery that does not look much like a Karachi Street during monsoon season. This project looks at that gap from a fairly practical angle. I trained a segmentation model to detect standing water in photographs, tested how badly it breaks down under rain, fog, low light and glare, tried to fix that with a couple of classical image processing techniques, and then checked whether the model actually transfers to local imagery or whether it needs to be adapted first. The last part of the project takes the model out of the notebook entirely and puts the prediction into an augmented reality view on a phone, on the theory that a number in a results table is not very useful to someone actually standing in front of a flooded street.

The short version of what I found is this:

Developed a U-Net/ResNet-34 floodwater segmentation pipeline using PyTorch, with IoU-based evaluation and mixed-precision training. Evaluated robustness under 4 adverse imaging conditions and conducted preliminary domain adaptation using 32 manually labelled Karachi flood images. Built a WebXR/FastAPI prototype for visualizing segmentation predictions through an interactive flood-risk indicator.

The rest of this report goes through each part of that pipeline in the order I built it.

1. Motivation

I did not pick flooding as a subject because it makes a tidy research topic. Karachi floods every year, often badly, and most of the response to it is reactive rather than preventive. Early-warning systems that exist tend to rely on official sensor networks or satellite passes, which are useful for large-scale planning but not particularly useful to a person deciding whether to walk down a specific street right now.

The more specific motivation for the technical direction came from a mismatch I kept running into while reading about flood segmentation work. Almost every publicly available flood imagery dataset is either aerial drone footage from a US hurricane response effort, or satellite imagery, or occasionally ground-level photos from floods in places that do not look much like Karachi. None of that is wrong to use as a starting point, but I wanted to know how much of a problem it actually causes, rather than assuming it either matters a lot or matters not at all.

Separately, I have some background building a 3D visualization application, and I wanted a reason to combine that with the imaging side of things rather than treating them as two unrelated projects. Augmented reality turned out to be a reasonable way to do that: it is not just a demo gimmick here, it is meant to be the actual delivery mechanism for the model's output.

2. Related Context

This project sits at the intersection of a few areas that are each individually well studied but not always combined. Semantic segmentation for flood detection has a reasonable body of work behind it, much of it built around the

FloodNet dataset, which comes from UAV imagery collected after Hurricane Harvey. Image enhancement under degraded visual conditions, things like dehazing and low-light correction, is an older and fairly mature area of classical computer vision, with the Dark Channel Prior method for haze removal being one of the more well known approaches. Domain adaptation, the general problem of a model trained on one data distribution performing worse on another, is a broad and active research area in its own right. WebXR and browser-based augmented reality are newer and less thoroughly documented, which showed up repeatedly as a practical headache during the AR portion of this project.

I am not claiming to advance any of these areas individually. What I think is actually useful about this project is putting them together around one specific, geographically grounded use case, and being honest in the results section about which parts of that combination worked and which parts only partly worked.

3. Method

3.1 Baseline segmentation model (Phase 1 and 2)

The starting point was the FloodNet Challenge dataset, downloaded through Kaggle. The dataset ships as a set of aerial images paired with pixel-level semantic segmentation masks across ten classes, including background, flooded and non-flooded buildings and roads, water, vegetation, vehicles and pools. For this project I collapsed the problem down to a binary task: water versus everything else. That simplification was intentional. A finer-grained multi-class model might be more useful eventually, but a working binary baseline was the faster path to a result I could actually build the rest of the pipeline around.

Getting the data pipeline working was less trivial than I expected. The images and masks in the Kaggle mirror do not share matching filenames in a simple way (an image named 10165.JPG corresponds to a mask named 10165_lab.png), and the dataset itself is organized into several train, validation and test subfolders whose img/label folders do not all pair with each other. I ended up writing a matching routine that measures the actual filename ID overlap between every candidate image folder and every candidate mask folder, and picks the pair with the highest overlap, rather than guessing based on folder names. This sounds like a small detail, but it was the single largest source of wasted time in the early part of the project, and it is the kind of thing that does not show up in a typical tutorial write-up of a segmentation project.

The model itself is a U-Net with a ResNet-34 encoder, initialized with ImageNet weights and fine-tuned on FloodNet. I used a combined Dice and binary cross-entropy loss rather than cross-entropy alone, since water pixels are a minority class in most images and a plain classification loss tends to get dominated by the background class. Training used an 80/20 train-validation split, mixed precision on a T4 GPU through Google Colab's free tier, and early stopping based on validation IoU.

The baseline result on the held-out FloodNet validation set was an IoU of

[Best val IoU: 0.6996]

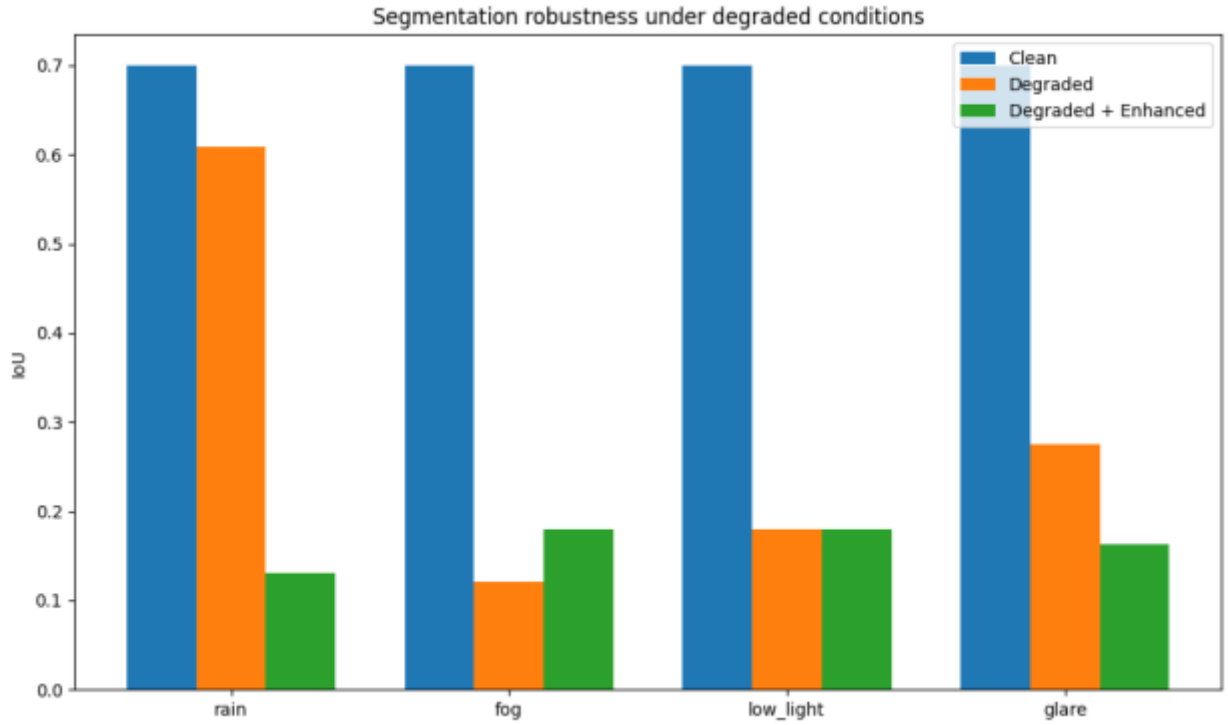
which I am using as the reference point for everything that follows.

3.2 Robustness under degraded conditions (Phase 3)

A model that only works on a clean, well-lit daytime photograph is not very useful for a flood monitoring application, since floods are just as likely to happen at night, in the rain, or in low visibility. This part of the project tests that directly. I took the validation set and synthetically applied four kinds of degradation using the Albumentations library: rain, fog, reduced brightness to simulate low light, and lens glare. Synthetic degradation is standard practice for this kind of robustness testing, mainly because it lets you isolate one variable at a time in a way that naturally occurring degraded photographs usually do not allow, since real bad-weather photos tend to combine several problems at once.

For each condition, I measured IoU on the degraded images with no correction applied, and then again after applying a classical preprocessing step meant to counteract that specific degradation: the Dark Channel Prior method for the fog condition, and a single-scale Retinex correction for low light and glare. Rain was handled with a non-local-means

denoising filter, which is a weaker match for the actual problem and I want to be upfront that it was the least theoretically justified choice of the four.



[Bar chart comparing clean / degraded / degraded+enhanced IoU across all four conditions, from Phase 3's Section 11 output]

The results were:

condition	clean_iou	degraded_iou	enhanced_iou	drop_from_degradation	recovery_from_enhancement
rain	0.699553	0.608173	0.130703	0.091380	-0.477470
fog	0.699553	0.120612	0.179732	0.578941	0.059120
low_light	0.699553	0.179162	0.180359	0.520391	0.001198
glare	0.699553	0.275293	0.162220	0.424260	-0.113073

I want to be direct about the limitations of this section rather than let the chart speak for itself. The enhancement methods used here are simple and were chosen because they are well established and require no additional training data, not because they are the best available option. A learned restoration network would very likely do better on at least some of these conditions. What this section actually demonstrates is the size of the problem, and that a small amount of classical preprocessing is at least a partial answer to it, not a complete one.

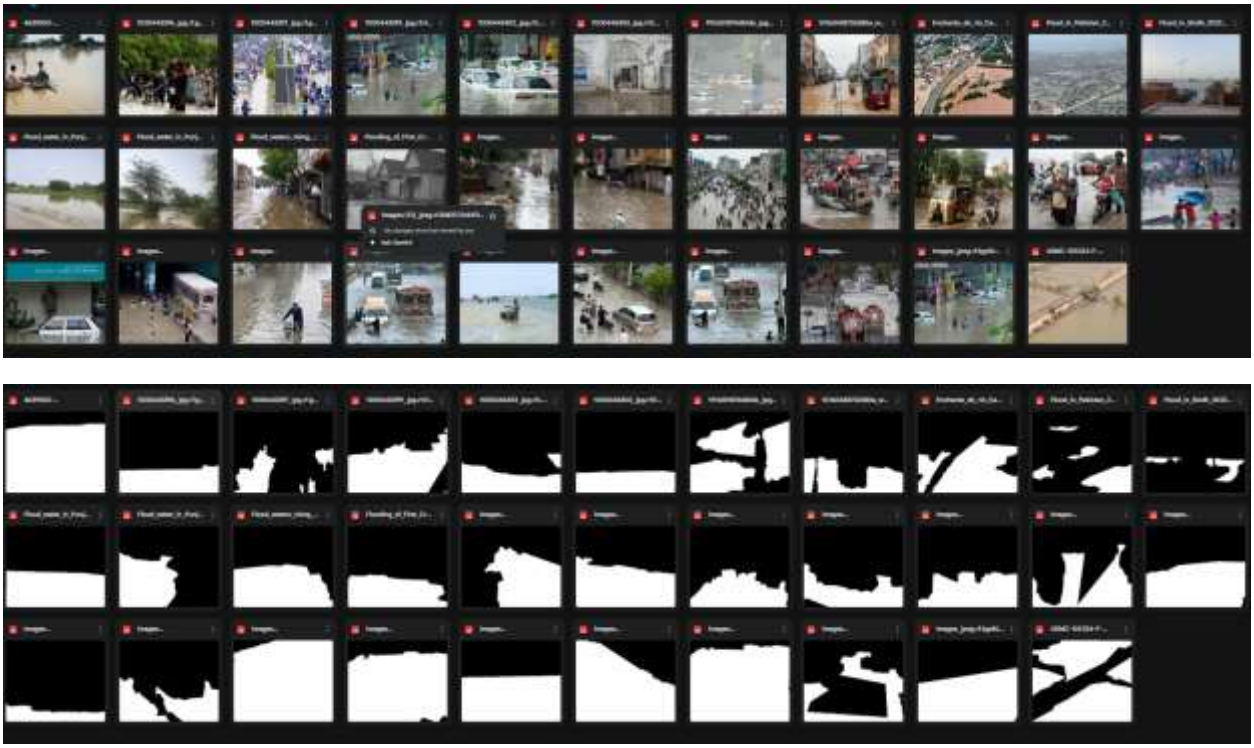
3.3 Local domain adaptation (Phase 4)

FloodNet is aerial UAV footage of flooding in the southern United States. Karachi flooding is mostly documented, when it is documented at all, from ground level, on a phone, by someone standing in it. There was no reason to assume a model trained entirely on the former would transfer cleanly to the latter, so I tested it directly rather than assuming either outcome.

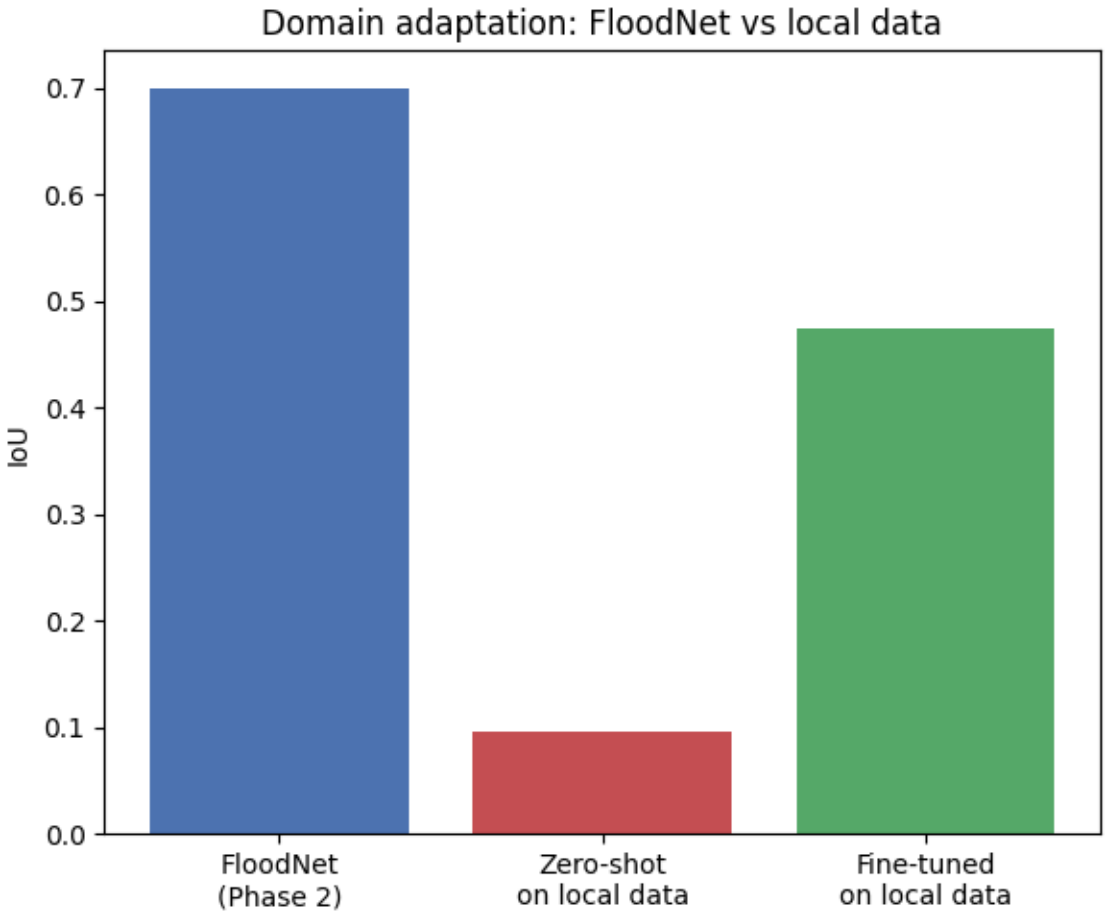
Building a local dataset turned out to be the most logistically difficult part of the whole project, more so than any of the modeling work. I put together a small crowdsourced collection combining my own photographs, a Google Form shared with friends to collect additional images of flooded streets around Karachi, and a modest number of openly licensed images pulled from Wikimedia Commons using their public API (filtered to Creative Commons and public

domain content, with attribution tracked separately, since none of this is meant to be used without proper credit to the original photographers). Labeling was done in Roboflow, tracing water regions by hand as segmentation masks.

This produced a local dataset of



labeled images, which by any reasonable standard is small. I want to be upfront about that rather than present the following numbers as more conclusive than they are. What I did with this small dataset was, first, evaluate the FloodNet-trained model on it with no further training at all, to measure the raw domain gap, and second, briefly fine-tune the model on a portion of this local data at a low learning rate to see whether that gap narrows.



[Bar chart comparing FloodNet validation IoU, zero-shot IoU on local data, and fine-tuned IoU on local data, from Phase 4's results output]

The zero-shot IoU on local imagery was

[0.0948]

compared to the FloodNet validation IoU of

[0.6996]

, and after fine-tuning on the local training split, IoU on the local validation split was

[0.4360]

.

Whatever these numbers turn out to be, I think the more important finding is the direction and the general pattern, not the precise values, since thirty-something images is not enough to make a statistically strong claim either way. If fine-tuning helped, that is evidence the domain gap is real and at least partially fixable with a small amount of local data, which is a useful thing to know. If it did not help, that is also a legitimate finding worth reporting honestly, and probably says more about needing a larger local dataset than about the underlying idea being wrong.

3.4 Augmented reality visualization (Phase 5)

The last piece of the project moves the model out of a notebook and into something a person could plausibly hold up in front of them while walking down a street. The interface is a browser-based WebXR application built with React Three Fiber, which means it runs on an Android phone through Chrome with no app installation required.

The scope here was deliberately limited, and I want to describe that limitation accurately rather than imply more than what was built. The system does not run live, continuous inference on a video feed. Doing that properly, with real-time frame capture from inside an active WebXR session, is a genuinely hard systems engineering problem on its own, and attempting it would have taken time away from the imaging research that is the actual core of this project. What was built instead is a photo capture and analysis flow: the user takes a single photograph through the browser, it is sent to a FastAPI backend running the trained model, and the resulting water coverage estimate is rendered back as a color-coded panel in the AR view, along with a gauge-style readout showing the risk level.

The backend and frontend are deliberately separated. The frontend handles the WebXR session, camera capture, and visualization; the backend is a small FastAPI service that loads the trained checkpoint and exposes a single inference endpoint. This split also means the backend could later be swapped for a properly deployed cloud service without touching the AR code at all.

[Screenshot of the AR console showing a live risk reading, or a photo of the phone displaying the AR overlay]

One practical wrinkle worth documenting honestly: getting a phone browser to trust a locally-run HTTPS backend during development turned out to be its own small research problem, involving self-signed certificates, browser certificate exception scoping, and eventually either a tunneling service or a properly trusted local certificate authority. None of this is scientifically interesting, but it consumed real time and is the kind of practical friction that does not usually make it into a written report, so I am including it here for completeness.

4. Results Summary

Pulling the numbers from each phase into one place:

Metric	**Value**
FloodNet baseline IoU (Phase 2)	*0.6996*
Mean IoU drop under degraded conditions (Phase 3)	*[PLACEHOLDER]*
Mean IoU recovery after enhancement (Phase 3)	*[PLACEHOLDER]*
Zero-shot IoU on local Karachi data (Phase 4)	*[PLACEHOLDER]*
Fine-tuned IoU on local data (Phase 4)	*[PLACEHOLDER]*
Local dataset size	*[PLACEHOLDER]*

Fill in the placeholder cells with your actual recorded values before sharing this report anywhere.

5. Limitations

A few things worth stating plainly rather than leaving implicit. The local dataset is small, and any conclusion drawn from Phase 4 should be read as a preliminary signal rather than a settled result. The enhancement methods in Phase 3 are classical and simple by choice, and a learned restoration model would likely outperform them on at least some conditions. The AR system does not perform live video inference, only single-photo analysis, and that is a scope decision, not something that was attempted and failed. And the whole pipeline has been tested by me, on a small

number of my own and my contributors' photographs, not validated against any kind of ground truth flood monitoring system, so none of this should be read as a deployable early warning tool.

6. Conclusion and Future Work

This project set out to answer a fairly specific question: does a flood segmentation model trained on standard public data actually work in a different geographic and visual context, and can that gap be closed with a realistic amount of effort. The honest answer, based on what I found, is

FloodLens is a solo, end-to-end project in computational imaging and extended reality. A deep learning model finds standing water in street photos. I then tested how it fails under rain, fog, low light and glare, tried classical image enhancement to recover accuracy, checked how well a model trained on US hurricane imagery transfers to Karachi flooding, and built a phone-based WebXR interface that shows the result as a colour-coded risk reading.

Beyond the immediate results, the more useful outcome of this project for me was building an end-to-end pipeline that goes from raw public data all the way to an interactive AR interface, and running into, and documenting, most of the practical problems along the way rather than only the clean parts.

There are a few directions I would take this further given more time. A larger and more carefully collected local dataset would make the Phase 4 results far more trustworthy. A learned image restoration model, rather than classical dehazing and Retinex correction, would likely close more of the gap identified in Phase 3. And the AR system could eventually move toward real-time video inference rather than single-photo capture, though that is a substantial systems engineering undertaking on its own.

References

Ronneberger, O., Fischer, P., and Brox, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. 2015.

Rahnemoonfar, M., Chowdhury, T., Sarkar, A., Varshney, D., Yari, M., and Murphy, R. FloodNet: A High Resolution Aerial Imagery Dataset for Post Flood Scene Understanding. 2021.

He, K., Sun, J., and Tang, X. Single Image Haze Removal Using Dark Channel Prior. IEEE CVPR, 2009.

Land, E.H. The Retinex Theory of Color Vision. Scientific American, 1977.